

Spécification et preuve de programmes

Solution du devoir 2 : spécifications et preuves avec micro-C

Alain Giorgetti

Licence d'Informatique de l'Université de Franche-Comté 2024-25

Ce devoir doit être rendu sous la forme d'un unique fichier texte *NomPrenom.c*, où *NomPrenom* sont les 8 premières lettres de votre nom de famille, suivies des 8 premières lettres de votre prénom, sans espaces ni accents. Ce fichier doit commencer par un commentaire entre */** et **/* indiquant vos nom et prénom. Ensuite, des commentaires entre */** et **/* doivent indiquer le numéro de chaque question traitée, par exemple

```
/* Exercice 2 question 3 */
```

juste avant la réponse à la question 3 de l'exercice 2.

La preuve et l'exécution du contenu complet de ce fichier doivent être possibles avec l'interface de micro-C en ligne. Dans ce but, les parties incorrectes ou incomplètes doivent être entourées par */** et **/*.

Lire l'énoncé jusqu'à la fin avant de commencer à traiter les exercices.

1 Logique du premier ordre (4 points)

Soit f et g deux fonctions sur le type `int` des entiers C. On dit que la fonction f est *un inverse à gauche* de la fonction g si $f(g(x)) = x$ pour tout entier x .

Si la fonction f est un inverse à gauche de la fonction g , alors la fonction g est *injective*, c'est-à-dire que $g(y) = g(z)$ implique $y = z$ pour tous les entiers y et z .

1. Formaliser ce théorème par un lemme micro-C, après avoir déclaré les fonctions f et g .

(3 points)

```
/*@
function int f(int i);
function int g(int i);

lemma implies: (forall x. f(g(x)) == x)
  -> (forall x. forall y. g(x) == g(y) -> x == y);
*/
```

2. Vérifier et confirmer par une phrase que l'interface en ligne de Why3 démontre automatiquement ce lemme. Si cette preuve échoue, expliquer pourquoi.

(1 point) L'interface en ligne de Why3 démontre automatiquement ce lemme.

2 Variant (4 points)

1. Donnez un exemple de fonction C avec une boucle `while` qui n'admet aucun variant correct, en justifiant votre réponse.

(1 point) La fonction suivante étant réduite à la boucle infinie, elle ne termine jamais, donc sa boucle n'admet aucun variant.

```
void f1() {  
    while (1) {  
    }  
}
```

2. Proposer un variant, selon la syntaxe micro-C, pour démontrer que la fonction suivante termine. Démontrer cette terminaison avec l'interface en ligne de Why3. Préciser s'il est nécessaire ou pas d'ajouter un invariant pour que cette démonstration de terminaison soit automatique.

```
int f(int n) {  
    while (1) {  
        n = n - 1;  
        if (n <= 0) break;  
    }  
    return(n);  
}
```

(1 point) Le code suivant propose un variant micro-C qui permet de démontrer automatiquement que la fonction termine, sans qu'il soit nécessaire d'ajouter un invariant.

```
int f2(int n) {  
    while (1) {  
        /*@ variant n; */  
        n = n - 1;  
        if (n <= 0) break;  
    }  
    return(n);  
}
```

3. Ajouter à cette fonction la précondition $n \geq 1$, selon la syntaxe micro-C. En supposant que cette précondition est vraie, proposer la postcondition micro-C la plus précise possible pour la valeur retournée par cette fonction. Proposer un invariant de boucle permettant de démontrer cette postcondition avec l'interface en ligne de Why3.

Le code suivant propose (1 point) la postcondition micro-C la plus précise possible pour la valeur retournée par cette fonction, et un invariant (1 point) suffisant pour que la preuve de la postcondition soit automatique avec l'interface en ligne de Why3.

```
int f3ngeq1(int n) {
  /*@ requires n >= 1;
    ensures result == 0; */
  while (1) {
    /*@ invariant n >= 1;
      variant n; */
    n = n - 1;
    if (n <= 0) break;
  }
  return(n);
}
```

3 Décalage circulaire en micro-C (12 points)

Le décalage circulaire dans un tableau d'entiers consiste à décaler chaque valeur dans la case suivante, sauf la dernière valeur, qui devient la valeur d'indice 0. Par exemple, le décalage circulaire du tableau

0	1	2	3	4	5	6
42	5	23	17	0	89	37

est le tableau

0	1	2	3	4	5	6
37	42	5	23	17	0	89

```
void shift (int a[], int n) {
  /*@ requires length(a) == n > 0; */
  int last = a[n-1];
  for (int k = n-1; k > 0; k--)
    a[k] = ..;
  a[..] = last;
}
```

Listing 1: Code à reproduire et à compléter.

1. Reproduire et compléter le code C du listing 1, pour que la fonction `shift` implémente le décalage circulaire dans le tableau non vide `a`, à l'aide d'une seule boucle `for`.

(2 points : 1 point par pointillé correctement complété)

```
void shift (int a[], int n) {
  int last = a[n-1];
  for (int k = n-1; k > 0; k--)
    a[k] = a[k-1];
  a[0] = last;
}
```

2. Proposer des invariants de boucle (en syntaxe micro-C) permettant de démontrer que tous les accès à des éléments du tableau dans la boucle de la fonction `shift` s'effectuent dans les limites 0 et `n-1` des indices du tableau.

Vérifier et confirmer par une phrase que l'interface en ligne de Why3 démontre automatiquement ces invariants.

(1 point)

```
/*@ invariant 0 <= k < n;
```

(1 point) L'interface en ligne de Why3 démontre automatiquement ces invariants.

3. Proposer un variant de boucle permettant de justifier la terminaison de la boucle de la fonction `shift`.

Vérifier et confirmer par une phrase que l'interface en ligne de Why3 démontre automatiquement cette terminaison.

(1 point)

```
variant k; */
```

(1 point) Avec ce variant, l'interface en ligne de Why3 démontre automatiquement cette terminaison.

4. Proposer des postconditions micro-C qui spécifient le décalage des valeurs du tableau.

(2 points : 1 point par postcondition)

```
ensures a[0] == old(a[n-1]);  
ensures (forall j. 1 <= j < n -> a[j] == old(a[j-1])); */
```

5. Proposer des invariants de boucle suffisamment précis pour permettre la preuve automatique des postconditions de la question 4.

Vérifier et confirmer par une phrase que l'interface en ligne de Why3 démontre automatiquement ces invariants et qu'ils sont assez précis pour démontrer les postconditions.

Indications : Proposer un premier invariant pour `a[0]`, un deuxième invariant pour le sous-tableau `a[1..k]` et un troisième invariant pour le sous-tableau `a[k+1..n-1]`. Dans un invariant, l'expression `old(e)` a la même signification que dans une postcondition : elle désigne la valeur de l'expression `e` avant l'exécution de la fonction. Si nécessaire, augmenter la valeur de `First-attempt step limit` dans les paramètres de l'interface en ligne de Why3.

(3 points : 1 point par invariant)

```
invariant a[0] == old(a[0]);  
invariant (forall j. 0 < j <= k -> a[j] == old(a[j]));  
invariant (forall j. k < j < n -> a[j] == old(a[j-1]));
```

(1 point) En augmentant à 1000 la valeur de `First-attempt step limit` dans les paramètres de l'interface en ligne de Why3, la démonstration de ces invariants est automatique et ils permettent de démontrer les postconditions.